ncsim: A Lightweight Discrete-Event Simulator for Networked Computing with 802.11 WiFi Models

Bhaskar Krishnamachari*, Maya Gutierrez*, and Jared Coleman*†
*University of Southern California
bkrishna@usc.edu, mayaguti@usc.edu
†Loyola Marymount University
jared.coleman@lmu.edu

***Abstract*—We present `ncsim`, a deterministic discrete-event simulator for evaluating DAG task scheduling algorithms on static wireless edge networks with IEEE 802.11 interference. `ncsim` models directed acyclic graph (DAG) workflows, multi-hop routing with bandwidth sharing, and IEEE 802.11 WiFi links with a physically-grounded CSMA/CA interference model based on Bianchi's analysis, including contention and hidden terminal SINR degradation. The simulator supports pluggable scheduling algorithms (including HEFT and CPOP via the SAGA library) and three routing strategies. Implemented in approximately 5,500 lines of Python with YAML-driven scenario specification, `ncsim` fills a gap between heavyweight network simulators like ns-3 and task-centric simulators like SimGrid, enabling rapid exploration of how wireless contention affects compute placement decisions in 802.11 edge deployments. We validate the WiFi model against analytical predictions across nine internal experiments (all matching within 1% tolerance) and externally against Bianchi's published results (matching within 0.003%). Using `ncsim`, we conduct a 108-run experimental study demonstrating that, under these modeled conditions, ignoring wireless interference leads to not just inaccurate makespan predictions but systematically wrong scheduler choices in 27.8% of the tested scenarios, with performance regret reaching 168%—underscoring the need for interference-aware simulation in wireless edge computing research.**

I. Introduction

Edge and fog computing architectures distribute computation across heterogeneous nodes connected by networks of varying capacity and reliability [1]. Effective task scheduling in these environments requires co-optimizing compute placement and network transport: a task placement that minimizes computation time may incur excessive data transfer costs, while a network-aware placement may underutilize fast processors.

Existing simulation tools address parts of this problem. Network simulators such as ns-3 [2] and OMNeT++ [3] provide detailed protocol stacks but are heavyweight, complex to configure, and lack native support for DAG-based task scheduling. Conversely, task scheduling simulators such as SimGrid [4], WRENCH [5], and CloudSim [6] model workflows and compute resources but typically abstract the network as point-to-point links with fixed bandwidths, omitting wireless contention, interference, and multi-hop effects. Edge-specific tools like iFogSim [7] and EdgeCloudSim [8] add domain-specific features but use simplified wireless models that do not capture the distance-dependent rates, CSMA contention, or hidden terminal interference that characterize real 802.11 deployments.

This gap matters more than one might expect. In a 108-run experimental study that we conduct using `ncsim`, we find that the scheduler which appears optimal under an interference-free model is *dominated* under realistic CSMA/CA conditions in 27.8% of scenarios, with makespan errors exceeding $94\times$. The sophisticated HEFT scheduler, optimized under a model that ignores wireless contention, can produce *worse* outcomes than a simple round-robin baseline. These findings underscore that realistic wireless modeling is not merely a refinement—it is essential for meaningful evaluation of edge computing scheduling algorithms.

We present `ncsim`, a lightweight, flow-level, static-topology discrete-event simulator that bridges this gap. `ncsim` combines DAG workflow scheduling with physically-grounded 802.11 WiFi models in a single, easy-to-use Python package. The contributions are demonstrated through a case study on small-to-medium 802.11 grid networks (up to 16 nodes) with static topologies:

1) A modular simulation architecture with abstract base classes for schedulers, routing models, interference models, link models, and queue models, enabling pluggable experimentation.
2) An 802.11 WiFi model incorporating log-distance path loss, MCS rate adaptation (802.11n/ac/ax), conflict graph construction, CSMA/CA contention via Bianchi's analysis, and hidden terminal SINR degradation.
3) Multi-hop routing with dynamic bandwidth sharing and three routing strategies (direct, widest-path, shortest-path).
4) Comprehensive validation: nine internal experiments achieving exact match with analytical predictions, plus external validation against Bianchi's published results [9] (within 0.003%).
5) Experimental demonstration—via a 108-run scheduler comparison and sensitivity analyses—that interference-free models lead to systematically wrong scheduling decisions, motivating the need for tools like `ncsim`.
6) An open-source implementation in $\sim$5,500 lines of Python with YAML-driven scenarios, JSONL trace output, and a React/D3 visualization frontend.

The remainder of this paper is organized as follows. Sec-

tion II surveys related work in DAG scheduling algorithms, edge orchestration systems, and simulation tools, positioning `ncsim` in the landscape. Section III describes the modular architecture and its design rationale. Section IV presents the system model, including the physically-grounded WiFi equations. Section V details the discrete-event simulation engine. Section VI validates the WiFi model through both internal analytical experiments and external reproduction of Bianchi's published results. Section VII presents the experimental evaluation, including routing comparison, interference impact analysis, a 108-run scheduler comparison revealing rank inversions, and sensitivity analyses. Finally, Section VIII discusses implications and concludes with directions for future work.

II. Related Work

We situate `ncsim` at the intersection of two research threads: (i) DAG scheduling algorithms and edge orchestration systems, and (ii) simulation tools that model wireless network effects. We survey both and highlight the gap that motivates our work. Table I compares simulation tools.

Classic DAG scheduling heuristics. DAG scheduling has a long history in parallel and distributed computing, with many heuristics built around the list-scheduling paradigm [11], [12]. Dynamic-level scheduling (DLS) accounts for interprocessor communication and heterogeneous interconnect constraints via dynamically updated task–processor priorities [13]. Among the most widely used baselines, HEFT and CPOP introduced rank-based task prioritization and greedy resource selection for heterogeneous processors with communication costs, and remain common reference points [10]. However, their standard formulations assume a largely stable platform with known execution and communication costs—a poor fit for wireless/edge deployments where load, link rates, and node availability vary over time.

Benchmarking and parametric scheduler design. A persistent challenge in this area is that comparative conclusions depend strongly on benchmark selection and implementation details. Coleman and Krishnamachari introduce SAGA, a modular open-source library for comparing task-graph schedulers, and PISA, a simulated-annealing adversarial search that identifies instances where one scheduler maximally underperforms another [14], [15]. Their results show that algorithms with similar average performance can diverge dramatically under adversarially chosen instances. Complementary parametric scheduling work decomposes list schedulers into interchangeable components, enabling systematic ablations across 72 hybrid variants [16]. These tools provide a stronger methodological foundation for robustness claims—especially important for wireless/IoT settings where typical instances may not resemble the stress cases encountered under interference or congestion.

Dispersed and edge execution systems. Beyond algorithm design, several systems integrate DAG scheduling with profiling and orchestration. Jupiter is an open-source networked computing architecture that orchestrates DAG task graphs using container-based deployment with both centralized (HEFT-style) and decentralized scheduling, reporting experiments on geographically distributed cloud nodes and Raspberry Pi clusters [17]. Tactical Jupiter adapts this to the tactical edge, addressing intermittent connectivity, bandwidth variability, and node attrition with disruption handling and ML-based task completion time estimation [18]. Throughput-oriented extensions target pipelined DAG workloads in dispersed settings [19]. These efforts emphasize that scheduling quality depends not only on the heuristic but also on profiling fidelity, control-plane overhead, and resilience to network dynamics.

DAG scheduling in wireless/edge/IoT environments. A growing body of work targets edge environments with heterogeneous devices and unreliable participation. I-BOT studies orchestration on unmanaged edge devices where heterogeneity, sporadic exits, and interference among co-located tasks dominate [20]. IBDASH schedules DAG applications on mixed commercial and personal edge devices, jointly optimizing latency and failure probability with strategic replication and availability prediction [21]. M-TEC generalizes to multi-tier edge computing with explicit cost considerations and evaluation on a commercial CBRS 4G network [22]. On the platform side, LAVEA provides edge-first video analytics with task offloading optimization [23], Hetero-Edge targets real-time vision on heterogeneous edge clouds with latency-predicted scheduling [24], and Petrel combines sample-based load balancing with task-type-aware policies [25]. MEC-oriented DAG offloading work commonly models wireless links via abstract rates [26], with recent extensions for energy budgets [27] and security constraints [28]. The fog/edge computing context motivating these systems is surveyed in [29], [30], while coded computing offers complementary reliability via redundancy [31]. These works are particularly relevant for IoBT-like settings where reliability and disruption are non-negotiable; however, most assume abstract network models that do not capture wireless contention at the physical layer.

Network simulators. ns-3 [2] and OMNeT++ [3] provide detailed WiFi, LTE, and TCP/IP protocol stacks with high-fidelity channel models, making them the standard choice for protocol-level wireless research. However, these tools are heavyweight: configuring an ns-3 scenario with custom DAG scheduling requires integrating external workflow engines, writing C++ glue code, and accepting simulation times that are orders of magnitude longer than the simulated duration. In our experience, a single 16-node scenario that `ncsim` completes in under 2 seconds would require several minutes in ns-3 with equivalent WiFi and application-layer setup. This computational cost makes parameter sweeps across hundreds of scheduler/routing/topology combinations prohibitive for researchers whose primary interest is scheduling algorithm design rather than protocol engineering.

Task scheduling simulators. SimGrid [4] and WRENCH [5] model distributed computing platforms with analytical network models and support DAG workflows natively. CloudSim [6] targets cloud computing with VM-level abstractions, and has been widely adopted for evaluating resource allocation

TABLE I: Simulator comparison. ncsim uniquely combines DAG-native scheduling with CSMA/CA and hidden terminal modeling at flow-level granularity.

Tool	DAG	WiFi	CSMA	Hid. Term.	Pkt-Level	Sweep
ns-3 [2]	✗	✓	✓	✓	✓	High
OMNeT++ [3]	✗	✓	✓	✓	✓	High
SimGrid [4]	✓	✗	✗	✗	✗	Low
WRENCH [5]	✓	✗	✗	✗	✗	Low
CloudSim [6]	✓	✗	✗	✗	✗	Low
iFogSim [7]	✓	✗	✗	✗	✗	Med
EdgeCloudSim [8]	✗	✓	✗	✗	✗	Med
ncsim	✓	✓	✓	✓	✗	**Low**

policies. These tools model networks as links with fixed or analytically-derived bandwidths, without wireless contention, hidden terminal effects, or carrier-sense-based spatial reuse. This abstraction is appropriate for wired data center networks where bandwidth is dedicated and predictable. However, in wireless edge deployments, the effective bandwidth of a link depends on which other links are simultaneously active—a dependency that fixed-bandwidth models cannot capture. As we demonstrate in Section VII, this omission can lead to not merely inaccurate predictions but systematically wrong conclusions about which scheduling algorithm is superior.

Edge/fog-specific simulators. iFogSim [7] extends CloudSim for fog computing topologies, adding latency-sensitive application placement. EdgeCloudSim [8] adds mobility and simplified WLAN models, typically using constant bandwidth with optional capacity degradation under load. While these tools capture some aspects of edge deployments, their wireless models do not account for distance-dependent rates, CSMA/CA contention dynamics, or hidden terminal interference—the phenomena that, as we show, most strongly affect scheduling outcomes in multi-hop wireless networks.

Positioning of ncsim. ncsim occupies a position between these tool classes: it combines DAG-aware task scheduling with physically-grounded 802.11 wireless models (SNR-based MCS rate adaptation, Bianchi MAC efficiency, SINR-based hidden terminal modeling) in a lightweight Python package. Unlike ns-3, ncsim does not simulate packet-level protocol stacks, which keeps simulation times tractable for large parameter sweeps. Unlike SimGrid and WRENCH, ncsim does not abstract away wireless interference, which preserves the scheduling-relevant effects of contention and hidden terminals. Unlike the DAG scheduling and orchestration systems surveyed above—which either assume abstract network models or rely on physical testbeds—ncsim provides a controlled simulation environment where wireless effects can be systematically varied and their impact on scheduling quality precisely measured. Table I summarizes this positioning.

ncsim is thus well-suited for rapid exploration of scheduling algorithms in wireless edge computing scenarios where both compute placement and wireless network effects matter.

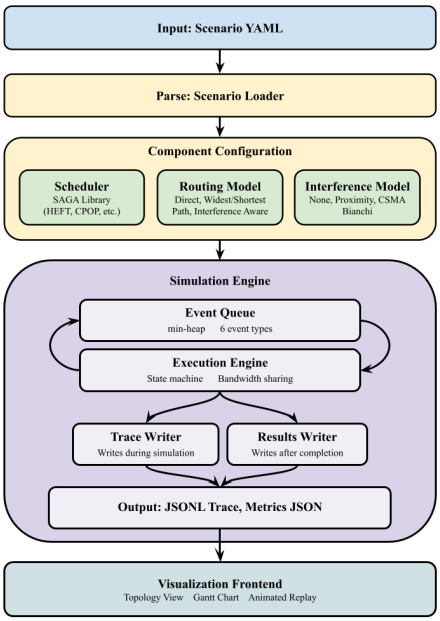

Fig. 1: High-level architecture of ncsim.

III. Architecture

In this section, we describe the architecture of ncsim, which is organized around a layered discrete-event simulation (DES) engine with pluggable components for scheduling, routing, and interference modeling. Figure 1 illustrates the data flow from scenario specification through simulation to output traces.

A. Design Goals

The design of ncsim is guided by four principles, each motivated by the requirements of scheduling research in wireless edge computing.

Determinism. Given the same scenario and random seed, ncsim produces bit-identical results across runs and platforms. We achieve this by rounding all floating-point simulation times to microsecond precision (10^{-6} s), which prevents the accumulation of platform-dependent rounding errors that can reorder near-simultaneous events. Determinism is essential for reproducible scheduling comparisons: if two schedulers produce makespans of 12.37 s and 12.39 s, the ranking must not change when the experiment is re-run on a different machine.

Modularity. Six abstract base classes (ABCs) define clean extension points: `Scheduler`, `RoutingModel`, `InterferenceModel`, `LinkModel`, `QueueModel`, and `DAGSource`. We chose this design because scheduling research often requires comparing algorithms across identical scenarios—modularity ensures that swapping a scheduler does not require changes to the network model, and vice versa. New strategies can be added by implementing a single ABC without modifying the core simulation engine.

Listing 1: Minimal WiFi scenario (abridged).

```yaml
scenario:
  network:
    nodes:
      - {id: n0, compute_capacity: 1000,
         position: {x: 0, y: 0}}
      - {id: n1, compute_capacity: 1000,
         position: {x: 30, y: 0}}
    links:
      - {id: l01, from: n0, to: n1, latency: 0.0}
  dags:
    - id: dag_1
      inject_at: 0.0
      tasks:
        - {id: T0, compute_cost: 10, pinned_to: n0}
        - {id: T1, compute_cost: 10, pinned_to: n1}
      edges:
        - {from: T0, to: T1, data_size: 50}
  config:
    scheduler: round_robin
    seed: 42
    interference: csma_bianchi
    rf:
      tx_power_dBm: 20
      freq_ghz: 5.0
      path_loss_exponent: 3.0
      wifi_standard: "ax"
```

Simplicity. Scenarios are specified in YAML files that declaratively describe the network topology, DAG workflows, and simulation parameters. The entire simulator is implemented in approximately 5,500 lines of Python with minimal dependencies (only the optional SAGA library for HEFT/CPOP scheduling). We deliberately chose Python over C++ or Java to lower the barrier for researchers who wish to extend the simulator or inspect its internals. Listing 1 shows a minimal WiFi scenario; note that omitting explicit `bandwidth` from links causes `ncsim` to derive rates from the `rf` configuration block.

Physical grounding. Wireless link bandwidths are derived from RF parameters (transmit power, frequency, path loss exponent) rather than specified as arbitrary constants. This ensures that link rates are physically consistent with node positions—a property that is essential for meaningful interference modeling, since contention and hidden terminal effects depend on the spatial relationships among transmitters and receivers.

B. Design Trade-offs

`ncsim` intentionally omits several features to maintain its lightweight, fast character while retaining the key phenomena—contention, hidden terminals, MCS adaptation—that affect scheduling decisions:

- **No packet-level simulation.** There is no TCP/IP stack or per-packet event processing; transfers are modeled as bulk flows with analytically-derived rates.
- **No physical-layer channel dynamics.** Fading, OFDM subcarrier allocation, and time-varying channel states are not modeled; path loss is static and deterministic (with optional log-normal shadow fading).
- **No mobility.** The topology is static; node positions are fixed for the duration of a simulation run.

TABLE II: Scope of validity: phenomena modeled by `ncsim` and their impact on scheduling studies.

Phenomenon	Status	Impact
CSMA/CA contention	Modeled (Bianchi)	High
Hidden terminals	Modeled (SINR)	High
MCS rate adaptation	Modeled (11n/ac/ax)	Medium
Path loss	Modeled (log-distance)	High
Pkt-level dynamics	Not modeled	Low[†]
TCP slow-start	Not modeled	Medium[‡]
Fading / mobility	Optional (log-normal)	Low[§]
Multi-slot parallelism	Not modeled	App-dependent

[†]For bulk transfers. [‡]For short flows. [§]For static deployments.

- **Single-slot compute.** Each node executes one task at a time (FIFO queuing); GPU-like multi-slot parallelism is left to future work.

These choices keep simulation runtimes on the order of seconds for networks of up to 16 nodes and 20-task DAGs, enabling rapid parameter sweeps that would be infeasible with packet-level simulators. Table II further summarizes the phenomena modeled and the boundaries of validity.

C. Modeling Scope and Validity

`ncsim`'s flow-level approach (Table II) models data transfers as bulk flows with analytically-derived rates rather than simulating individual packets. This abstraction is appropriate for the workloads studied here (10 MB payloads per edge), where steady-state throughput dominates and TCP slow-start transients are negligible. For workloads with many short transfers (e.g., sub-100 KB control messages or request-response patterns), the slow-start phase represents a significant fraction of each transfer's duration, and packet-level simulation would be warranted. The Bianchi model further assumes saturation conditions, meaning that every station always has a packet ready to transmit. Additionally, the hidden terminal model does not include capture effects: in real 802.11 radios, a sufficiently strong desired signal can be decoded even in the presence of weaker interferers, reducing hidden terminal impact. Together, the saturation and no-capture assumptions make `ncsim`'s interference-aware makespans conservative (i.e., pessimistic) upper bounds, particularly in the hidden terminal regime.

The grid topologies used throughout this paper are regular and symmetric; real deployments may have irregular node placements that create asymmetric interference patterns. Scaling to 50–100 node networks is computationally feasible with `ncsim` but was not explored here. Real wireless channels exhibit fading and mobility effects that could either smooth or amplify the interference effects; we use deterministic path loss throughout to isolate the effects of contention and hidden terminals.

D. Key Abstractions

Table III summarizes the six pluggable abstractions in `ncsim`.

TABLE III: Pluggable abstractions in `ncsim`.

ABC	Implementations
Scheduler	Manual, RoundRobin, HEFT, CPOP
RoutingModel	Direct, WidestPath, ShortestPath
InterferenceModel	None, CSMA Bianchi
LinkModel	Static (future: position-based, TDMA)
QueueModel	FIFO (future: priority, multi-slot)
DAGSource	Single, Multi

E. Visualization and Tooling

To support debugging and pedagogical use, `ncsim` includes a React/D3 web-based visualization frontend with three complementary views:

1) A **network topology view** that displays nodes with their compute capacities, links with their current bandwidths, and active transfers highlighted with color-coded flow indicators.
2) A **Gantt chart** showing task execution and data transfer timelines per node, making it possible to visually identify scheduling bottlenecks, idle periods, and contention-induced delays.
3) An **animated replay** with playback controls (play, pause, step, speed adjustment) that allows researchers to observe the dynamic evolution of the simulation, including the bandwidth recalculation cascades described in Section V.

The Gantt view is particularly useful for diagnosing the contention-induced idle periods and cascading delays revealed in Section VII.

Simulation output is written in JSONL trace format, with one JSON object per event, enabling integration with external analysis tools such as pandas, matplotlib, and custom post-processing scripts. The command-line interface supports batch execution with configurable scheduling, routing, and interference parameters, facilitating the automated parameter sweeps used throughout Section VII.

F. Research Questions Enabled

The architecture described above enables several classes of research questions that existing tools cannot address jointly: (1) How does wireless contention change which DAG scheduler is best (Section VII-D)? (2) When does single-flow optimal routing become counterproductive under multi-flow interference (Section VII-B)? (3) What are the parameter regimes where interference can safely be ignored versus where it is essential (Section VII-E)? (4) How does multi-tenant DAG injection scale under shared-medium contention (Section VII-E)?

IV. System Model

In this section, we present the formal models underlying `ncsim`: the network and compute infrastructure, the DAG workflow representation, the scheduling and routing frameworks, and the physically-grounded 802.11 WiFi model that captures contention and hidden terminal effects.

A. Network Model

A network consists of *nodes* and directed *links*. Each node v has a compute capacity C_v (compute units per second) and an optional 2D position (x_v, y_v) used for RF calculations. Each link ℓ from node u to node v has a bandwidth B_ℓ (MB/s) and latency L_ℓ (seconds). Topologies are specified in YAML and can be arbitrary directed graphs.

B. Compute Model

A task t has a compute cost w_t (in compute units). The execution time on node v is w_t/C_v. Each node processes one task at a time using FIFO queuing. Tasks follow a five-state lifecycle: Pending → Ready → Queued → Running → Completed.

C. DAG Workflow Model

Workflows are modeled as directed acyclic graphs (DAGs) where nodes represent tasks and edges represent data dependencies. Each edge $e = (t_i, t_j)$ carries a data payload of size d_e (in MB) that must be transferred from the node executing t_i to the node executing t_j before t_j can begin. If t_i and t_j are placed on the same node, the transfer is instantaneous (zero cost). DAGs are injected at specified simulation times.

D. Scheduling Framework

The scheduler receives a `NetworkSnapshot`—an immutable view of node capacities, link bandwidths, queue depths, and active transfers—and returns a `PlacementPlan` mapping each task to a node. `ncsim` includes four schedulers:

- **Manual:** Uses `pinned_to` annotations from the YAML.
- **RoundRobin:** Cycles through nodes, respecting pin constraints.
- **HEFT/CPOP:** Adapted from the SAGA scheduling library [10] via an adapter that constructs a fully-connected virtual network with actual bandwidths for connected pairs and near-zero bandwidth for unreachable pairs, causing the scheduler to naturally avoid infeasible placements.

E. Routing and Bandwidth Sharing

Three routing models determine the path for data transfers:

- **Direct:** Only allows transfers over explicit single-hop links.
- **WidestPath:** Modified Dijkstra maximizing bottleneck bandwidth.
- **ShortestPath:** Standard Dijkstra minimizing total latency.

Multi-hop transfers use store-and-forward semantics: the effective bandwidth is the minimum (bottleneck) across all links in the path, and latency is the sum. When multiple transfers share a link, each receives a fair share:

$$B_{\text{per-flow}}(\ell) = \frac{B_\ell \cdot f(\ell)}{N_\ell} \tag{1}$$

where $f(\ell)$ is the interference factor and N_ℓ is the number of concurrent flows on link ℓ. When a transfer starts or completes, all affected transfers are recalculated and their completion events rescheduled.

F. WiFi / 802.11 Model

The WiFi model replaces static link bandwidths with rates derived from RF parameters, and models contention and hidden terminal interference using physically-grounded analytical models. The following subsections formalize these mechanisms.

1) Physical Layer: The received power at distance d uses log-distance path loss:

$$\mathrm{PL}(d) = \mathrm{PL}(d_0) + 10\, n\, \log_{10}\left(\frac{d}{d_0}\right) \tag{2}$$

$$P_{\mathrm{rx}}(d) = P_{\mathrm{tx}} - \mathrm{PL}(d) \tag{3}$$

where $\mathrm{PL}(d_0)$ is the Friis free-space loss at reference distance $d_0 = 1\,\mathrm{m}$, n is the path loss exponent, and P_{tx} is the transmit power. The signal-to-noise ratio is $\mathrm{SNR} = P_{\mathrm{rx}} - N_0$ (in dB), where N_0 is the noise floor. The path loss exponent n ranges from 2 (free-space) to ≥ 3.5 (dense indoor).

The PHY data rate is selected from the appropriate 802.11 MCS table based on the computed SNR:

$$R_\ell = \max\{r_k : \mathrm{SNR}_\ell \geq \mathrm{SNR}_{\min,k}\} \tag{4}$$

This selects the highest-rate MCS whose minimum SNR requirement is met, producing a discrete rate staircase—a key feature that continuous-rate models fail to capture.

2) Carrier Sensing and Conflict Graph: The carrier sensing range d_{CS} is the maximum distance at which a transmission triggers the clear channel assessment (CCA) mechanism:

$$d_{\mathrm{CS}} = d_0 \cdot 10^{\frac{P_{\mathrm{tx}} - \theta_{\mathrm{CCA}} - \mathrm{PL}(d_0)}{10\, n}} \tag{5}$$

The conflict graph $G_C = (L, E_C)$ is defined over links. Without RTS/CTS, two links conflict if either transmitter can sense any node of the other link. With RTS/CTS, any node of one link sensing any node of the other creates a conflict.

3) CSMA Bianchi Model (`csma_bianchi`): The dynamic model separates two interference mechanisms. Let $\mathcal{A}$ be the set of currently active links, $\mathcal{C}_\ell = \mathcal{A} \cap \mathcal{N}(\ell)$ the active contending neighbors, and $\mathcal{H}_\ell = (\mathcal{A} \setminus \mathcal{N}(\ell)) \setminus \{\ell\}$ the active hidden terminals.

a) Contention factor.: The number of contending stations is $n = 1 + |\mathcal{C}_\ell|$. Using Bianchi's saturation throughput analysis [9], the MAC efficiency $\eta(n)$ is computed from the coupled equations for transmission probability τ and collision probability p:

$$\tau = \frac{2(1-2p)}{(1-2p)(W_{\min}+1) + p\, W_{\min}(1-(2p)^m)} \tag{6}$$

$$p = 1 - (1-\tau)^{n-1} \tag{7}$$

with minimum contention window $W_{\min} = 16$ and maximum backoff stage $m = 6$. We solve numerically via bisection (Section VI-B) to obtain the equilibrium (τ^*, p^*). The MAC efficiency is then:

$$\eta(n) = \frac{P_{\mathrm{success}} \cdot T_{\mathrm{success}}}{\mathbb{E}[T_{\mathrm{slot}}]} \tag{8}$$

where P_{success}, T_{success}, and $\mathbb{E}[T_{\mathrm{slot}}]$ are the probability, duration, and expected slot time for successful transmissions, respectively. Each contending station receives a fair share $\eta(n)/n$ of the channel capacity.

b) Hidden terminal factor.: Hidden terminals transmit simultaneously with link ℓ, causing interference at the receiver. The SINR is:

$$\mathrm{SINR}_\ell = 10 \log_{10}\left(\frac{P_{\mathrm{rx}}^{(\mathrm{lin})}}{N_0^{(\mathrm{lin})} + \sum_{j \in \mathcal{H}_\ell} P_j^{(\mathrm{lin})}}\right) \tag{9}$$

In 802.11, each frame is transmitted at a pre-selected MCS and either decoded successfully or lost entirely—there is no fallback to a lower MCS during reception [32], [33]. The MCS rate-selection thresholds include operating margins (implementation loss, fading margin, AGC settling) that are not required for frame decoding. The *decode threshold* for a given MCS is therefore lower than the selection threshold by a capture margin Δ:

$$\theta_{\mathrm{decode}} = \theta_{\mathrm{select}} - \Delta \tag{10}$$

We use $\Delta = 5\,\mathrm{dB}$, consistent with the gap between ns-3's `IdealWifiManager` selection and its `TableBasedErrorRateModel` decode thresholds, and with measured capture thresholds of 4–10 dB in the literature [34]. The hidden terminal factor is:

$$f_{\mathrm{HT}}(\ell) = \begin{cases} 1.0 & \text{if } \mathrm{SINR}_\ell \geq \theta_{\mathrm{decode}} \\ 0.01 & \text{otherwise (frame failure)} \end{cases} \tag{11}$$

c) Combined factor.: The total interference degradation combines both effects multiplicatively:

$$\boxed{f(\ell) = f_{\mathrm{HT}}(\ell) \cdot \frac{\eta(n)}{n}} \tag{12}$$

clamped to $[0.01, 1.0]$. When transfers start or complete, `ncsim` recalculates $f(\ell)$ for all active transfers sharing a conflict graph neighbor with the affected link, and reschedules their completion events accordingly.

V. Simulation Engine

In this section, we describe the discrete-event simulation engine that forms the core of `ncsim`. The engine orchestrates the interplay between task execution, data transfer, and interference dynamics through an event-driven architecture.

A. Event Processing

`ncsim` uses an event-driven DES with a min-heap priority queue. Six event types are processed in priority order at each simulation time:

1) DagInject — Invokes scheduler, initializes task states
2) TaskComplete — Frees node, triggers output transfers
3) TransferComplete — Delivers data, checks readiness
4) TaskReady — Starts task or enqueues if node busy
5) TaskStart — Begins execution, schedules completion
6) TransferStart — Acquires link bandwidth, schedules completion

The priority ordering among event types at the same simulation time is critical for causal correctness. Completions are processed before starts, and task completions before transfer completions, so that freed nodes can accept new work

immediately. Consider a scenario where task T_3 completes on node v at time t, and transfer $X_{2\to4}$ also completes at time t, making task T_4 ready. The ordering ensures that T_3's completion frees node v before T_4 attempts to start, so T_4 can potentially be placed on the now-idle node v rather than being queued.

Events are cancellable via lazy deletion: when bandwidth recalculation invalidates a previously-scheduled completion time, the stale event is marked invalid and a new event is inserted. Stale events are discarded when they reach the head of the queue. All times are rounded to microsecond precision (10^{-6} s) to ensure deterministic behavior across platforms.

B. Bandwidth Recalculation Cascade

The most technically subtle aspect of the simulation engine is the bandwidth recalculation cascade that occurs whenever a transfer starts or completes. This cascade is the mechanism by which `ncsim` captures the dynamic nature of wireless interference: the effective bandwidth of a link depends on which other links are simultaneously active, and this set changes throughout the simulation as transfers begin and end.

When a transfer starts or completes on link ℓ, the engine identifies all active transfers whose links share a conflict graph neighbor with ℓ. For each affected transfer, the engine performs three steps: (1) recompute the number of contending neighbors $|\mathcal{C}_\ell|$ and hidden terminals $|\mathcal{H}_\ell|$; (2) recalculate the combined interference factor $f(\ell)$ from Equation (12); and (3) update the bottleneck bandwidth for the end-to-end path. If the new bandwidth differs from the previously assumed value, the engine cancels the stale completion event and schedules a new one based on the remaining data and updated bandwidth.

To illustrate, consider a 3×3 grid where transfer X_1 is active on link $(0,0)\to(1,0)$ with an expected completion at $t = 5.0$ s. At $t = 2.0$ s, transfer X_2 starts on the adjacent link $(1,0)\to(2,0)$, which is a conflict graph neighbor of X_1's link. The engine recalculates: X_1 now has one contending neighbor, so its interference factor drops from $f = 1.0$ (solo) to $f = \eta(2)/2 \approx 0.44$. With approximately 56% less effective bandwidth, the remaining data will take longer to transfer, so the engine cancels the $t = 5.0$ s completion event and schedules a new one at approximately $t = 8.8$ s. This recalculation ripples to any other active transfers that neighbor X_2's link, potentially triggering further reschedules.

In practice, the cascade is bounded by the conflict graph neighborhood size, which ranges from 4 to 12 for our grid topologies. The overall complexity per simulation is $O(E \cdot K \cdot \log N)$ where E is the total number of events, K is the average neighborhood size, and N is the maximum number of pending events. Each simulation in our 108-run study completes in under 2 seconds on commodity hardware.

VI. Validation

In this section, we validate the correctness of `ncsim`'s WiFi model through three complementary approaches. First, we verify internal self-consistency by comparing simulation output against analytical predictions computed directly from the RF and MAC equations across nine controlled experiments. Second, we validate externally by reproducing the numerical results from Bianchi's seminal paper on 802.11 DCF analysis [9], using his exact parameters. Third, we cross-validate against ns-3 [2] packet-level simulations to confirm that the combined model produces realistic throughput predictions. Together, these validations establish that `ncsim`'s interference model is self-consistent, faithful to the published analytical framework, and agrees with packet-level ground truth.

TABLE IV: Experiment 1: Link length vs. data rate (802.11ax, 5 GHz).

Dist(m)	SNR(dB)	MCS	Pred(MB/s)	Sim(MB/s)
1	68.58	11	17.925	17.925
12	36.20	9	14.338	14.338
30	24.27	5	8.600	8.600
50	17.61	3	4.300	4.300
75	12.33	2	3.225	3.225
105	7.94	0	1.075	1.075
140	4.19	n/a	0.000	0.000

A. Internal Analytical Validation

We designed nine experiments, each isolating a specific aspect of the WiFi model, to verify that `ncsim`'s simulation output matches analytical predictions computed directly from the RF and MAC equations. We use a 1% match tolerance, but in practice all nine experiments achieve *exact* match (zero error to the precision reported). We present three representative experiments in detail below; a fourth experiment (N-way Bianchi contention scaling) is presented alongside the external validation in Section VI-B. The remaining five experiments validate two-transmitter shared-receiver topologies, staggered transfer starts with dynamic bandwidth recalculation, per-link bandwidth sharing with multiple concurrent flows, five-link hidden terminal cascades with varying data sizes, and combined interference scenarios. All pass with zero error.

1) Experiment 1: Link Length vs. Data Rate: A single link between two nodes at varying distances from 1 m to 140 m. Table IV shows selected results. The simulated rate matches the analytically predicted MCS-selected rate at every distance. At 140 m the SNR (4.19 dB) falls below MCS 0 (5 dB threshold), correctly yielding zero rate. Figure 2 plots the characteristic staircase pattern: rate is constant within each MCS band and drops discretely at each SNR threshold.

2) Experiment 2: Parallel Link Separation: Two parallel 30 m links separated vertically by distances from 5 m to 200 m. Figure 3 illustrates the three interference regimes. At separation $\leq$ 70 m, all node pairs are within the carrier sensing range (71.2 m), so the links *contend* via Bianchi's model: each gets $\eta(2)/2 \approx 0.44$ of the base rate 8.6 MB/s = 3.79 MB/s. At separation $\geq$ 75 m, the links become hidden terminals, and SINR degradation determines the effective rate.

Notably, the effective rate *drops* at the CS range boundary—from 3.79 MB/s (contention) to 3.23 MB/s (hidden terminal at 75 m)—before recovering at larger separations. This non-monotonic behavior is the classic *hidden terminal effect*: within

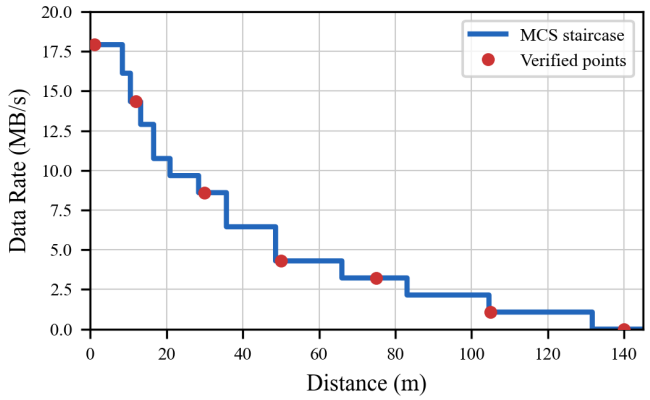

Fig. 2: Experiment 1: PHY data rate vs. link distance showing discrete MCS transitions (802.11ax, 5 GHz, n=3). Red dots mark experimentally verified data points from Table IV.

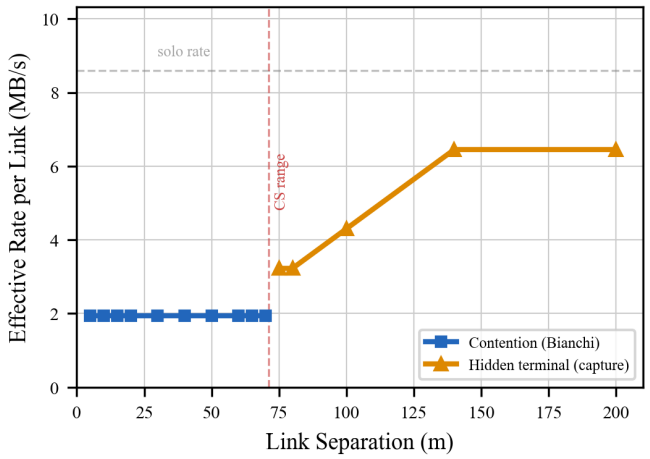

Fig. 3: Experiment 2: Effective per-link rate vs. separation for two parallel 30 m links. Below the CS range (71.2 m), links contend via Bianchi's model. Above it, hidden terminal SINR degradation causes a non-monotonic *dip*: the rate drops from 3.79 to 3.23 MB/s at the boundary because uncoordinated simultaneous transmissions degrade SINR more than fair time-sharing reduces throughput. Rate recovers as interference power falls with increasing distance.

sensing range, links coordinate via CSMA/CA and time-share the channel fairly (each receiving 44% of the full 8.6 MB/s); just outside sensing range, links transmit simultaneously without coordination, and the resulting mutual interference degrades each receiver's SINR enough to force a lower MCS rate (from MCS 5 to MCS 2). The hidden terminal regime is thus *worse* than fair contention sharing at close range. As separation increases further, interference power diminishes: SINR recovers to MCS 3 (4.3 MB/s) at 90 m and MCS 4 (6.45 MB/s) at 130 m. All 15 data points match predictions exactly.

3) Experiment 4: Three Parallel Links: This experiment extends the two-link topology of Experiment 2 to three parallel

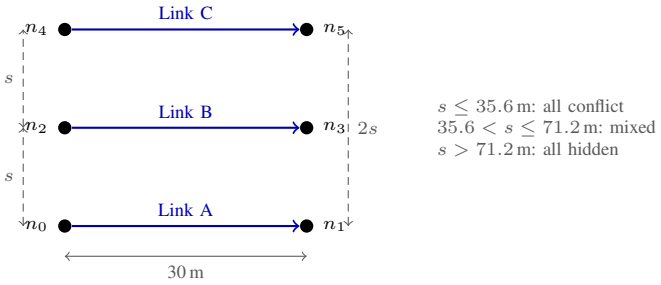

Fig. 4: Experiment 4 topology: three parallel 30 m links at variable separation s. As s increases, the interference regime transitions from all-conflict to mixed (adjacent pairs contend, outer pair A–C are hidden terminals) to all-hidden.

TABLE V: Experiment 4: Three parallel links at varying separation. In the mixed regime the middle link B contends with both neighbors while outer links A, C contend only with B and see each other as hidden terminals.

s(m)	Regime	Pred R_A	Sim R_A	Pred R_B	Sim R_B
10	all-conf	2.461	2.461	2.461	2.461
35	all-conf	2.461	2.461	2.461	2.461
40	mixed	1.861	1.861	2.461	2.461
50	mixed	2.174	2.174	2.461	2.461
70	mixed	2.840	2.840	2.720	2.720
75	all-hid	3.225	3.225	2.867	2.867
100	all-hid	4.300	4.300	3.822	3.822
150	all-hid	6.450	6.450	5.160	5.160

A=C by symmetry in all cases. Rates in MB/s.

30 m links placed at $y = 0$, $y = s$, and $y = 2s$ (links A, B, C), each carrying a 10 MB transfer. Figure 4 illustrates the topology with separation s as the independent variable.

As s increases, three interference regimes emerge: (i) *all-conflict* ($s \leq 35.6$ m): all three pairs are within d_{CS}, each link gets $R_{\text{base}} \cdot \eta(3)/3$; (ii) *mixed* ($35.6 < s \leq 71.2$ m): adjacent pairs (A–B, B–C) contend via Bianchi, while the outer pair (A–C at distance $2s$) are hidden terminals, creating asymmetric rates; and (iii) *all-hidden* ($s > 71.2$ m): no conflict-graph edges, each link sees the others purely as hidden terminals. Because A/C and B generally have different rates, one group finishes first, and the remaining links' interference conditions change. The analytical prediction accounts for this multi-phase behavior.

Table V shows the results. All 11 test points match exactly. In the mixed regime at s=70 m, an interesting crossover occurs where A/C become faster than B, because the outer links benefit from their mutual hidden-terminal distance while B contends with both neighbors. This experiment validates the model's ability to correctly compose Bianchi contention and SINR degradation in a single topology with asymmetric interference conditions and multi-phase dynamic recalculation.

4) Additional Validation Experiments: Five additional experiments validate: (3) two-transmitter shared-receiver topologies, (5) staggered transfer starts with dynamic recalculation, (6) per-link bandwidth sharing with multiple flows, (8) five-link hidden terminal cascades with different data sizes, and (9) combined

TABLE VI: Reproduction of Bianchi [9] Table III: normalized saturation throughput S for W=32, m=3.

n	Computed	Published	Rel. Error
2	0.847311	0.8473	0.001%
3	0.836828	0.8368	0.003%

bandwidth sharing with hidden terminal interference. All pass.

B. External Validation Against Bianchi (2000)

Internal validation confirms self-consistency but does not establish that our implementation of Bianchi's equations is correct. We therefore reproduce the numerical results from Bianchi's original paper [9] using his exact parameters: FHSS 1 Mbps basic access (no RTS/CTS), with slot duration 50 μs, SIFS 28 μs, DIFS 128 μs, propagation delay 1 μs, payload 8184 bits, MAC header 272 bits, PHY header 128 bits, and ACK 112 bits.

1) Table III Reproduction: Table VI compares our computed normalized saturation throughput S against the values published in Bianchi's Table III for W=32, m=3. At n=2, our solver computes $S = 0.847311$ versus the published 0.8473 (relative error 0.001%). At n=3, we compute $S = 0.836828$ versus 0.8368 (relative error 0.003%). These sub-basis-point differences are consistent with rounding in the published table.

Our solver for the coupled equations (Equations (6) and (7)) uses a bisection method rather than simple fixed-point iteration. Simple iteration can oscillate when the contention window is small or the number of stations is large, because the mapping can have a slope exceeding unity near the fixed point. Bisection is guaranteed to converge within $\lceil \log_2(1/\epsilon) \rceil$ iterations for any tolerance ϵ. We use $\epsilon = 10^{-12}$, requiring at most 40 bisection steps.

2) Figure 6 Reproduction: We also reproduce Bianchi's Figure 6 throughput curves for two configurations: W=32, m=5 ($\text{CW}_{\max}$=1024) and W=128, m=3 ($\text{CW}_{\max}$=1024). Figure 5 plots the computed normalized throughput S across the full range of station counts from n=5 to n=50. Table VII presents the numerical values underlying these curves.

Several features of these curves merit discussion. First, both configurations exhibit the expected monotonic decrease in throughput with increasing n: as more stations contend, collision overhead grows and the fraction of time spent on successful transmissions decreases. Second, the W=128 curve shows a slight non-monotonicity between n=5 ($S = 0.825$) and n=10 ($S = 0.826$). This occurs because the larger initial contention window is oversized for a small number of stations, causing excessive idle time; as n increases from 5 to 10, the window becomes better matched to the contention level before collision losses begin to dominate. This non-monotonicity is consistent with Bianchi's published results and would not be captured by simpler throughput models. Third, the gap between the two curves narrows at large n: at n=50, W=128 achieves $S = 0.725$ versus $S = 0.611$ for W=32 (a 19% advantage), whereas at n=5 the gap is only 2%. This confirms the well-known principle that larger contention windows provide greater benefit in high-contention regimes.

TABLE VII: Numerical values for Bianchi Figure 6 reproduction: normalized saturation throughput S for two (W, m) configurations.

n	S (W=32, m=5)	S (W=128, m=3)
5	0.8102	0.8250
10	0.7579	0.8263
15	0.7231	0.8130
20	0.6975	0.7981
25	0.6772	0.7837
30	0.6603	0.7702
35	0.6457	0.7577
40	0.6329	0.7461
45	0.6214	0.7353
50	0.6109	0.7252

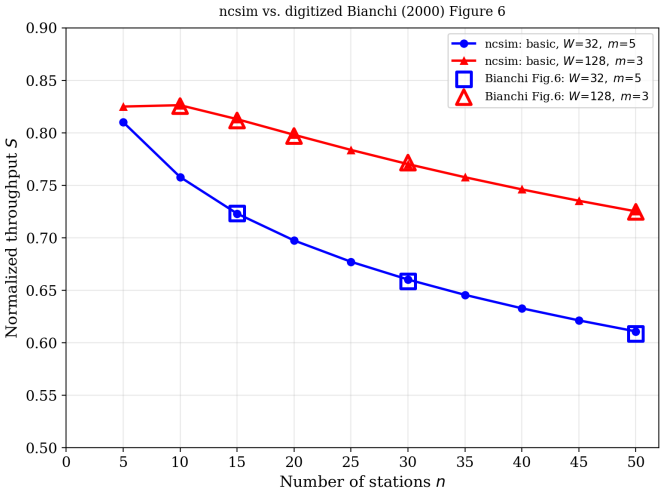

Fig. 5: Reproduction of Bianchi [9] Figure 6: normalized saturation throughput vs. number of stations for two (W, m) configurations. Computed using Bianchi's exact FHSS parameters (1 Mbps basic access, no RTS/CTS). The W=128 curve's slight non-monotonicity between n=5 and n=10 is consistent with Bianchi's published results. Data points digitized from Bianchi's original Figure 6 overlay our computed curves to within 0.34% relative error, confirming faithful reproduction of the analytical model.

3) N-Way Contention Scaling: As a further check that the Bianchi contention model scales correctly within the full `ncsim` simulation loop, we place $n \in \{2, \ldots, 8\}$ parallel 30 m links at 5 m vertical separation—all within carrier-sensing range, creating a complete conflict graph of size n. Each link carries a 10 MB transfer. The predicted per-link rate is $R = R_{\text{base}} \cdot \eta(n)/n = 8.6 \cdot \eta(n)/n$.

Table VIII shows all seven data points match exactly. The MAC efficiency $\eta(n)$ degrades gracefully from 0.881 at n=2 to 0.726 at n=8, while the per-link share $\eta(n)/n$ falls much faster due to the $1/n$ division. This confirms that the Bianchi solver, validated numerically against Table III and Figure 6 above, also produces correct throughput predictions when embedded in the

TABLE VIII: N-way Bianchi contention scaling (30 m links at 5 m separation). $\eta(n)$ is the Bianchi MAC efficiency.

n	$\eta(n)$	$\eta(n)/n$	**Pred(MB/s)**	**Sim(MB/s)**
2	0.881	0.440	3.787	3.787
3	0.859	0.286	2.461	2.461
4	0.837	0.209	1.799	1.799
5	0.818	0.164	1.407	1.407
6	0.802	0.134	1.149	1.149
7	0.788	0.113	0.968	0.968
8	0.726	0.091	0.780	0.780

full simulation engine with real RF parameters and dynamic recalculation.

C. Cross-Validation Against ns-3

The preceding validation establishes internal self-consistency and faithful reproduction of Bianchi's analytical results, but all comparisons are between analytical formulas—ncsim implements the same equations it validates against. To test whether the *combined* model behavior (path loss + MCS selection + CSMA/CA contention + hidden terminal capture effects) produces realistic throughput predictions, we cross-validate against ns-3 [2], a widely-used packet-level network simulator that models 802.11 at full protocol fidelity.

Both experiments use 802.11ax at 5 GHz on a 20 MHz channel with `ConstantRateWifiManager` at HE-MCS 5 (64-QAM, coding rate 2/3), TX power 20 dBm, CCA threshold −82 dBm, noise figure 6 dB (effective floor −95 dBm), $\text{CW}_{\min}$=16, $\text{CW}_{\max}$=1024, guard interval 3200 ns, 1 spatial stream, RTS/CTS off, and—critically—A-MPDU disabled to match Bianchi's single-frame assumption. The propagation model is log-distance with exponent 3.0 and Friis reference loss 46.4 dB at 1 m, identical to `ncsim`'s `RFConfig` defaults. Each ns-3 point represents the mean over 20 independent seeds with 30 s simulation time (first 2 s discarded as warmup).

1) Experiment 1: Contention Scaling: We place $n \in \{1, \ldots, 8\}$ co-located STA-AP link pairs at 30 m link length and 5 m vertical separation—all well within the 71.2 m carrier-sensing range, creating a complete conflict graph. Each STA sends saturated UDP traffic to its AP. Figure 6(a) overlays `ncsim`'s Bianchi prediction $R_{\text{base}} \cdot \eta(n)/n$ against the ns-3 per-station goodput.

`ncsim`'s Bianchi model—which accounts for PHY overhead (preamble, SIFS, ACK) in the efficiency computation—tracks ns-3 closely across all station counts. At n=1, the solo goodput predictions differ by only 0.3% (3.78 vs. 3.77 MB/s). The maximum error of 7.6% occurs at n=7, where the Bianchi fixed-point approximation slightly overestimates MAC efficiency. The mean error across all n is 4.0%. Both curves exhibit the same monotonic decay shape with Pearson correlation $r > 0.999$. All 95% confidence intervals are below ±0.02 MB/s, confirming that 30 s of saturated traffic reaches steady state.

2) Experiment 2: Separation Sweep: Two parallel 30 m links are placed at vertical separation $s \in \{10, 20, \ldots, 200\}$ m. As s increases beyond the 71.2 m carrier-sensing boundary, the links transition from Bianchi contention (symmetric time-sharing at $\eta(2)/2$) to the hidden terminal regime, where simultaneous transmissions may corrupt frames at the receiver. `ncsim`'s capture-threshold model (Equation (10)) predicts a *throughput dip* at the CS boundary—per-link rate drops from 1.94 MB/s (contention) to 1.50 MB/s (hidden terminal frame loss) before recovering sharply at $s = 120$ m when the interferer's power falls below the capture threshold and frames are decoded successfully at the full solo rate (3.78 MB/s). Figure 6(b) overlays ns-3 results.

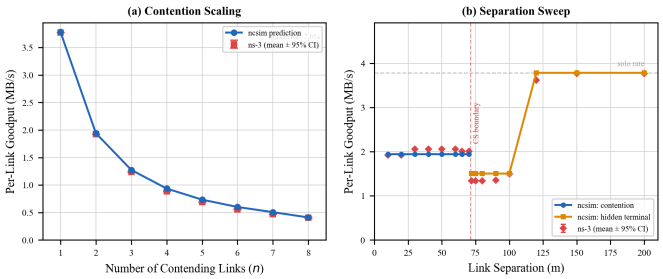

Fig. 6: Cross-validation of `ncsim` against ns-3 packet-level simulation. (a) Per-station saturated goodput under n-way contention: `ncsim` tracks ns-3 within 0.3–7.6% across all station counts ($r > 0.999$). (b) Two-link throughput vs. separation: ns-3 confirms the throughput dip at the CS boundary (71.2 m, dashed red) and recovery at large separations. The capture-threshold model matches within 0.3–11% across all regimes. All parameters aligned per Table IX; each ns-3 point: mean ± 95% CI over 20 seeds.

TABLE IX: Aligned parameters for ns-3 cross-validation.

Parameter	**ncsim**	**ns-3**
TX power	20 dBm	`TxPowerStart/End = 20`
Frequency	5 GHz	Band 5 GHz, ch. 36
Channel width	20 MHz	`ChannelSettings`
Path loss exp.	3.0	`Exponent = 3.0`
Ref. loss (1 m)	46.4 dB	`ReferenceLoss = 46.4`
Noise floor	−95 dBm	`RxNoiseFigure = 6`
CCA threshold	−82 dBm	`CcaEdThreshold = -82`
$\text{CW}_{\min}$	16	`MinCw = 15`
$\text{CW}_{\max}$	1024	`MaxCw = 1023`
A-MPDU	disabled	`MaxAmpduSize = 0`
Guard interval	3200 ns	`GuardInterval = 3200`

ns-3 confirms the qualitative behavior across all regimes. In the contention zone ($s \leq 70$ m), ns-3 goodput is stable at 1.92–2.06 MB/s (1–6% from `ncsim`). At the CS boundary ($s = 72$ m), ns-3 exhibits a throughput dip to 1.34 MB/s—confirming `ncsim`'s predicted dip direction and location (11% error). In the hidden terminal zone ($s = 72$–100 m), `ncsim`'s capture-threshold model predicts 1.50 MB/s and ns-3 measures 1.34–1.50 MB/s (0.5–11% error). At large separations ($s \geq 120$ m) where interference power falls below the capture threshold, both models show recovery: `ncsim` predicts the full solo rate of 3.78 MB/s and ns-3 reaches 3.62–3.77 MB/s (0.3–4.4% error).

The ns-3 simulation scripts, Dockerfile for reproducible builds, and all raw results are included in the paper's supplementary repository under `paper/ns3/`.

TABLE X: Scheduler inputs. No scheduler observes interference.

Scheduler	Planner Inputs	Network View	Intf?
Manual	Pin annotations	N/A	No
RoundRobin	Node list	N/A	No
HEFT	DAG, C_v, B_ℓ	Full virtual net†	No
CPOP	DAG, C_v, B_ℓ	Full virtual net†	No

†Actual BW for reachable pairs; 0.001 MB/s for unreachable.

VII. Experimental Evaluation

Having validated the correctness of `ncsim`'s WiFi model, we now turn to the central question that motivates this work: does ignoring wireless interference merely produce inaccurate makespan estimates, or does it lead to fundamentally wrong conclusions about which scheduling and routing policies are superior? We evaluate `ncsim` through four sets of experiments that progressively build the case. First, we compare routing strategies to demonstrate that the choice of routing algorithm interacts non-trivially with interference. Second, we quantify the magnitude of interference impact on makespan predictions. Third, we execute a full 108-run factorial comparison revealing that interference changes *which scheduler wins* in over a quarter of scenarios. Finally, sensitivity analyses identify the parameter regimes where interference is most consequential.

All experiments use 802.11ax at 5 GHz with a path loss exponent of $n = 3$ (except where explicitly varied), heterogeneous compute capacities (80–300 units/s), 10 MB data payloads per DAG edge, and deterministic seed 42.

A. Scheduler Information and Fairness

A fair comparison between interference models requires that the schedulers receive identical inputs in both cases. Table X summarizes what each scheduler knows at planning time. Crucially, no scheduler sees the conflict graph, contention factors, or hidden terminal structure—these are runtime phenomena computed by the interference model *after* the schedule has been committed. HEFT and CPOP receive WiFi-derived PHY rates (the same MCS-selected bandwidths under both `none` and `csma_bianchi`), but they cannot anticipate the dynamic bandwidth degradation that contention will impose during execution.

The comparison between `none` and `csma_bianchi` is therefore fair: both present an identical `NetworkSnapshot` to the scheduler, containing the same WiFi-derived link bandwidths. The only difference is whether the simulation engine applies contention and hidden terminal degradation at runtime. Any difference in makespan is thus attributable solely to wireless interference effects that the scheduler did not—and could not—anticipate.

B. Routing Strategy Comparison

We compare widest-path and shortest-path routing under HEFT scheduling with the `csma_bianchi` interference model across grid mesh networks of three sizes (2×2, 3×3, 4×4) and three DAG complexities (5, 10, and 20 tasks).

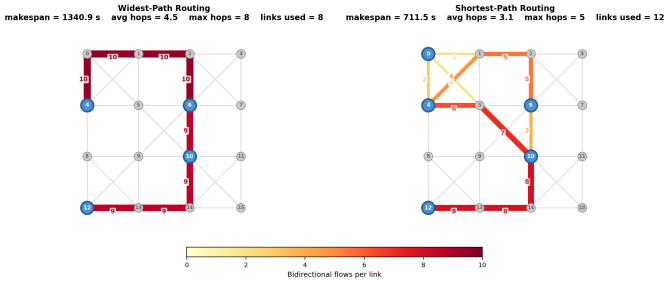

Fig. 7: Link utilization under widest-path (left) vs. shortest-path (right) routing on a 4×4 grid mesh with a 20-task DAG. Widest-path concentrates 9–10 flows on an 8-hop perimeter corridor; shortest-path spreads traffic across 12 links with shorter routes, reducing makespan by 1.9× under CSMA contention.

All links are wireless with 40 m grid spacing. Nodes have heterogeneous compute capacities (80–300 units/s). Table XI shows the results.

We evaluate these two strategies on a 4×4 grid wireless mesh network (16 nodes, 68 bidirectional links) running a 20-task DAG with 5-stage pipeline structure. Nodes are spaced 40 m apart with heterogeneous compute capacities (80–300 compute units/s). All tasks have compute cost 500 and all edges carry 10 MB data transfers. The HEFT scheduler assigns tasks to nodes; the routing strategy determines the multi-hop path for each inter-node data transfer. WiFi link rates follow the 802.11ax MCS table based on SNR at each link distance, and all links contend for airtime under the CSMA/Bianchi MAC model.

Figure 7 shows the resulting link utilization for each strategy. Link color and thickness indicate the number of bidirectional data flows traversing each link. Widest-path routing funnels nearly all traffic through an 8-hop perimeter corridor (nodes 0–1–2–6–10–14–13–12), loading each link with 9–10 flows and leaving the right half of the network idle. Shortest-path routing distributes traffic across 12 links with shorter routes (avg 3.1 hops vs. 4.5), spreading load more evenly. Under CSMA contention, widest-path's flow concentration produces severe airtime competition: its makespan is 1341 s compared to 712 s for shortest-path (1.9× slower). The result illustrates that single-flow optimal routing (maximizing bottleneck bandwidth) becomes counterproductive when multiple concurrent transfers compete for shared wireless airtime.

Key findings. Shortest-path routing wins in 7 of 9 configurations with 2 ties. The advantage is most pronounced in the 4×4 grid, where widest-path routing selects longer paths with higher bottleneck bandwidth but incurs substantially more wireless interference due to multi-hop transmissions traversing more contention domains. In the 4×4 grid with 5 tasks, widest-path achieves a makespan of 315.2 s compared to 12.9 s for shortest-path—a 24× difference—because the widest paths route transfers through 3–4 hops across the full grid diameter, creating overlapping contention domains that reduce effective bandwidth to a fraction of the solo rate.

TABLE XI: Routing comparison: makespan (seconds) for widest-path (W) vs. shortest-path (S) under HEFT with `csma_bianchi`.

Network	DAG	W (s)	S (s)	Winner
2×2 (4 nodes)	5 tasks	15.38	15.38	Tie
	10 tasks	32.60	32.50	Shortest
	20 tasks	88.13	64.51	Shortest
3×3 (9 nodes)	5 tasks	19.06	19.06	Tie
	10 tasks	79.88	35.16	Shortest
	20 tasks	142.94	96.70	Shortest
4×4 (16 nodes)	5 tasks	315.20	12.88	Shortest
	10 tasks	1396.9	827.23	Shortest
	20 tasks	1703.4	794.39	Shortest

TABLE XII: Interference impact: makespan (seconds) without interference vs. with `csma_bianchi`, using shortest-path routing and HEFT scheduling.

Network	DAG	None (s)	Bianchi (s)	Slowdown
2×2 (4n, 12 links)	5 tasks	11.43	15.38	+35%
	10 tasks	18.76	32.50	+73%
	20 tasks	27.50	64.51	+135%
3×3 (9n, 32 links)	5 tasks	8.43	19.06	+126%
	10 tasks	12.76	35.16	+176%
	20 tasks	17.11	96.70	+465%
4×4 (16n, 68 links)	5 tasks	8.93	12.88	+44%
	10 tasks	15.02	827.23	+5406%
	20 tasks	21.75	794.39	+3553%

In the 2×2 grid, the small topology limits contention: both routing strategies produce essentially identical makespans for 5 tasks (tie) and 10 tasks (0.3% difference), while shortest-path wins convincingly for 20 tasks (64.5 s vs. 88.1 s). Such topology-dependent scaling underscore the value of simulating the full compute-network interaction rather than selecting routing strategies based on bandwidth alone.

C. Interference Impact Analysis

We measure the impact of CSMA/CA interference by running each of the nine grid-network scenarios with the `csma_bianchi` model versus no interference (`none`), both using shortest-path routing and HEFT. As detailed in Section VII-A, both modes present identical `NetworkSnapshot` inputs to the scheduler; the only difference is the presence or absence of runtime contention and hidden terminal effects. Table XII summarizes the results.

Key findings. Interference impact ranges from 35% to over 5400% across all configurations. Several patterns emerge:

- **No configuration escapes contention entirely.** Even the smallest scenario (2×2, 5 tasks) shows a 35% slowdown, because the Bianchi MAC overhead reduces effective throughput whenever links share the channel.
- **Denser networks amplify interference.** The 3×3 grid (32 links) shows 126–465% slowdown, while the 4×4 grid (68 links) shows up to 5406%. More links create larger conflict neighborhoods.
- **Dense interference creates cascading delays.** In the 4×4 grid, HEFT distributes tasks across 16 nodes assuming uncontested bandwidth. During execution, many simultaneous transfers contend, reducing effective rates and causing HEFT's schedule to become severely suboptimal.

These results demonstrate that ignoring wireless interference in scheduling can lead to order-of-magnitude performance prediction errors, motivating the physically-grounded model in `ncsim`.

D. Scheduler Comparison and Rank Inversions

The interference impact results above suggest a deeper question: does ignoring interference merely produce inaccurate makespan predictions, or does it lead to wrong *choices* among schedulers? To answer this, we execute a full factorial comparison: 3 networks × 3 DAGs × 3 schedulers (HEFT, CPOP, RoundRobin) × 2 routings × 2 interference models = 108 simulation runs. All runs use 802.11ax at 5 GHz, heterogeneous compute capacities, and seed 42.

For each of the 18 (network, DAG, routing) triples, we identify the best scheduler under each interference model. A *rank inversion* occurs when the winning scheduler changes between the interference-free and interference-aware models. The *scheduling regret* is the relative performance penalty from deploying the interference-free-optimal scheduler in the presence of interference.

Table XIII presents the results. The key statistics are:

- **Rank inversion rate: 27.8%** (5 of 18 triples).
- **Mean relative regret: 20.2%** across all 18 triples.
- **Maximum relative regret: 168.3%** (4×4 grid, 10-task DAG, widest_path: HEFT chosen under "none" gives 1396.9 s, but round_robin achieves 520.6 s under csma_bianchi).

HEFT dominates under interference-free models. Under the "none" interference model, HEFT or CPOP wins in all 18 triples. This is expected: both algorithms optimize the critical path by distributing tasks across nodes to maximize parallelism, assuming uncontested bandwidth. The sophisticated scheduling logic of HEFT and CPOP provides clear advantages when the network behaves as these algorithms assume it will.

Round-robin wins in large networks under interference. In the 4 × 4 grid with csma_bianchi and widest-path routing, round-robin wins for all three DAG sizes—a striking reversal. The mechanism is clear: HEFT spreads tasks across many of the 16 available nodes to exploit parallelism, but this creates many concurrent multi-hop transfers that contend on shared wireless channels. Round-robin's simpler placement cycles through nodes in order, which tends to concentrate work on nearby nodes with fewer concurrent transfers and thus less contention.

To illustrate this mechanism concretely, consider the most dramatic rank inversion: the 4 × 4 grid with 10 tasks and widest-path routing. Under the interference-free model, HEFT achieves a makespan of 14.9 s by distributing tasks across many of the 16 nodes, creating multiple concurrent inter-node transfers that exploit the assumed link bandwidths. However,

TABLE XIII: Winning scheduler per (network, DAG, routing) triple under interference-free ("None") and csma_bianchi ("Bian.") models. Bold entries indicate rank inversions. Makespan in seconds shown in parentheses.

Network	DAG	Widest-path		Shortest-path	
		None	Bian.	None	Bian.
2×2	5 tasks	HEFT (11.4)	HEFT (15.4)	HEFT (11.4)	HEFT (15.4)
	10 tasks	HEFT (18.1)	HEFT (32.6)	HEFT (18.8)	HEFT (32.5)
	20 tasks	CPOP (26.2)	CPOP (78.1)	HEFT (27.5)	HEFT (64.5)
3×3	5 tasks	HEFT (8.4)	HEFT (19.1)	HEFT (8.4)	HEFT (19.1)
	10 tasks	CPOP (13.7)	CPOP (67.1)	HEFT (12.8)	HEFT (35.2)
	20 tasks	HEFT (15.7)	HEFT (142.9)	HEFT (17.1)	HEFT (96.7)
4×4	5 tasks	HEFT (8.2)	**RR (196.8)**	HEFT (8.9)	HEFT (12.9)
	10 tasks	HEFT (14.9)	**RR (520.6)**	HEFT (15.0)	**RR (417.1)**
	20 tasks	CPOP (22.3)	**RR (1694.7)**	CPOP (20.7)	**HEFT (794.4)**

under csma_bianchi, these concurrent transfers contend on overlapping conflict graph neighborhoods: each transfer's effective bandwidth drops to a fraction of its solo rate as the Bianchi model allocates channel time among contenders. The resulting makespan *explodes* to 1,396.9 s—a 94× blowup. In contrast, round-robin achieves 520.6 s under the same interference model. Although round-robin makes no attempt to optimize the critical path, its placement pattern generates fewer concurrent transfers, resulting in less severe contention. The scheduling regret—the penalty for choosing HEFT based on the interference-free evaluation—is 168%. A researcher evaluating these schedulers without interference modeling would confidently select HEFT, unaware that it produces 2.7× worse performance than the "naive" round-robin baseline.

Routing modulates the effect. Shortest-path routing mitigates rank inversions in some cases by reducing the number of hops per transfer and thus the number of contention domains traversed. The 4×4 / 5-task scenario shows a rank inversion under widest-path routing (60% regret) but not under shortest-path (zero regret). However, shortest-path does not eliminate inversions entirely: in the 4×4 / 10-task scenario, the inversion persists even with shortest-path routing (98% regret), because the DAG is large enough to generate concurrent transfers even along short paths.

Figure 8 visualizes the regret: for each triple, an arrow connects the oracle makespan (best scheduler under csma_bianchi) to the naive makespan (scheduler chosen under "none," run under csma_bianchi). The most dramatic cases occur in the 4×4 grid where HEFT's parallelism-maximizing placement creates dense contention.

These results demonstrate a key capability of `ncsim`: by jointly modeling scheduling and wireless interference, it reveals that interference-free evaluations can lead not merely to inaccurate estimates but to systematically wrong policy choices—a finding that would be invisible to tools that abstract away wireless contention.

E. Sensitivity Analysis

The results above demonstrate that wireless interference can dramatically affect scheduler rankings. However, a natural question arises: under what conditions is interference most consequential, and when can it safely be ignored? To address this question, we conduct two sensitivity sweeps that vary key parameters affecting interference severity: the communication-to-computation ratio (data size per DAG edge) and the level of concurrent workload (number of simultaneous DAGs).

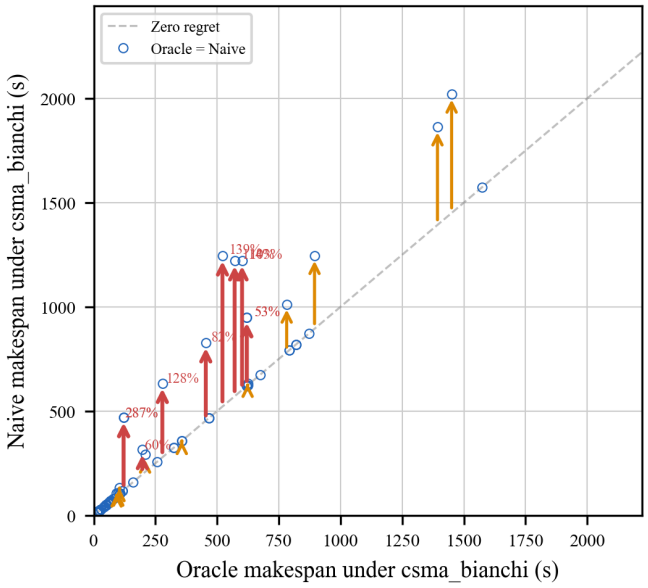

Fig. 8: Makespan regret scatter plot. Each arrow connects the oracle makespan (best scheduler under csma_bianchi) to the naive makespan (scheduler chosen under interference-free model, run under csma_bianchi). Red arrows indicate rank inversions; orange arrows indicate moderate regret. 5 of 18 triples exhibit rank inversions.

1) Communication-to-Computation Ratio: The communication-to-computation ratio (CCR) determines the relative importance of network transfer time versus compute time. We sweep data size per edge across $\{1, 2, 5, 10, 20, 50, 100\}$ MB on the 3×3 grid with HEFT and shortest-path routing, testing all three DAG types (5, 10, and 20 tasks). Figure 9 shows the interference slowdown as a function of data size.

The results reveal a non-monotonic pattern: slowdown peaks at intermediate CCR values (≈0.3–1.1) and decreases at both

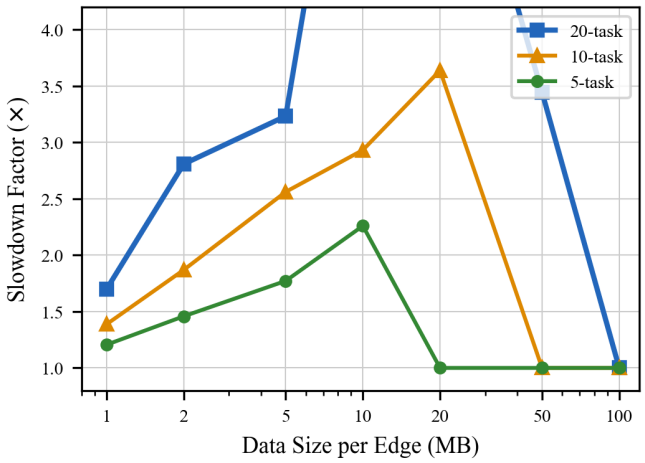

Fig. 9: Interference slowdown vs. data size per edge (3×3 grid, HEFT, shortest-path). Slowdown peaks at intermediate CCR values where transfers are long enough to contend but not so large that the scheduler collapses tasks onto one node.

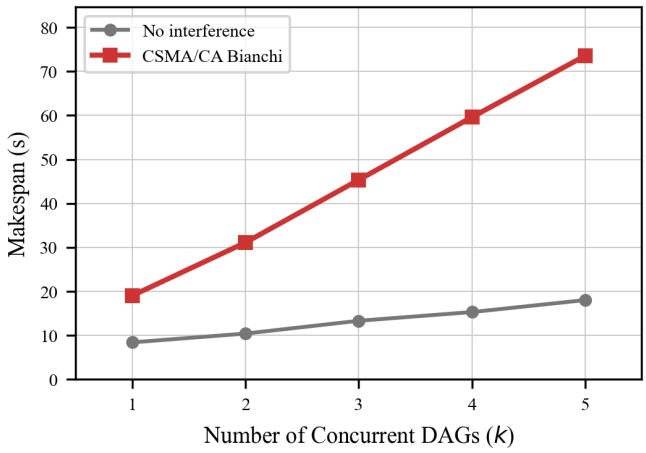

Fig. 10: Multi-DAG makespan scaling on 3×3 grid (5-task DAGs, HEFT, shortest-path). Contention amplifies super-linearly: the gap between the curves widens from 2.26× (k=1) to 4.08× (k=5).

extremes. The mechanism differs at each end. At low CCR (1–2 MB), transfers complete so quickly that the probability of temporal overlap between concurrent flows is small, and contention has little opportunity to develop. At very high CCR (50–100 MB), HEFT's scheduling logic recognizes the enormous transfer costs and collapses tasks onto fewer nodes to avoid inter-node communication entirely—a scheduling adaptation that eliminates contention by eliminating transfers. The interference "danger zone" lies in between, where transfers are long enough to overlap but the scheduler still distributes tasks across multiple nodes. For the 20-task DAG, the peak slowdown of 7.3× occurs at 10 MB per edge, corresponding to a CCR of approximately 0.5 (where transfer time and compute time are comparable).

2) Multi-DAG Contention: In practice, edge computing platforms rarely service a single workflow in isolation—multiple applications submit DAGs concurrently, and their transfers share the same wireless medium. To evaluate how `ncsim` captures this multi-tenant scenario, we inject $k \in \{1, 2, 3, 4, 5\}$ identical 5-task fork-join DAGs with 0.5 s stagger on the 3×3 grid (HEFT, shortest-path). Each DAG is scheduled independently, meaning that HEFT optimizes each workflow without knowledge of the others' transfers.

Figure 10 shows that contention scaling is super-linear under `csma_bianchi`. Without interference, k=5 DAGs produce 2.14× the makespan of k=1 (18.0 s vs. 8.4 s), reflecting the expected linear increase from staggered injection plus queuing at compute nodes. With interference, however, k=5 produces 3.86× the single-DAG makespan (73.5 s vs. 19.1 s). The gap between the two curves widens with each additional DAG: the slowdown factor grows from 2.26× at k=1 to 2.98× at k=2 to 4.08× at k=5, indicating a positive feedback loop. Each additional DAG contributes not only its own transfers to the contention pool but also prolongs the duration of existing transfers (through reduced effective bandwidth), which in turn increases the temporal overlap with subsequent DAGs. This super-linear scaling is invisible to interference-free models, which predict near-linear growth and would substantially underestimate the makespan of multi-tenant edge deployments.

TABLE XIV: Sensitivity analysis summary. Each row identifies the parameter regime where interference impact is most severe.

Experiment	Parameter	Danger Zone	Peak Effect
CCR	Data size (MB)	5–20 MB	7.3× at 10 MB
Multi-DAG	DAG count k	$k \geq 2$	4.1× gap at k=5

3) Sensitivity Summary: Table XIV consolidates the findings across both sensitivity dimensions. Taken together, these results delineate an *interference danger zone*—the conjunction of parameter settings under which interference effects are most severe and most likely to change scheduling conclusions. Wireless interference has the greatest impact—and is most likely to produce rank inversions—at moderate communication-to-computation ratios (5–20 MB payloads per edge) with multiple concurrent workflows ($k \geq 2$).

These conditions are not exotic. Data payloads of 5–20 MB per DAG edge are representative of computer vision pipelines (transferring image frames between processing stages) and sensor fusion applications (aggregating data from multiple modalities). Multiple concurrent workflows are the norm on shared edge platforms. In other words, the interference danger zone corresponds precisely to the deployment scenarios that motivate edge computing research—which means that tools ignoring interference are most unreliable in exactly the settings where they are most needed.

F. Scalability

Our experiments validate `ncsim` at scales up to 4×4 grids (16 nodes, 68 links) with 20-task DAGs. At these sizes,

individual runs complete in under 2 seconds on commodity hardware (single-threaded Python 3.11); the full 108-run factorial comparison completes in under 4 minutes. The main computational bottlenecks are: (1) conflict graph construction, which uses the Bron-Kerbosch algorithm (exact for ≤ 50 links, with a greedy approximation fallback for larger graphs); and (2) event queue size, which grows with concurrent transfers. Scaling to larger networks (e.g., 8×8 grids or 100+ node topologies) may require the greedy clique approximation and would benefit from a compiled-language event queue, but remains tractable for the scheduling studies `ncsim` targets.

VIII. Conclusion and Future Work

We presented `ncsim`, a lightweight, flow-level, static-topology discrete-event simulator that bridges the gap between heavyweight network simulators and task-centric scheduling tools by combining DAG workflow scheduling with physically-grounded 802.11 WiFi models in a single, easy-to-use Python package. The simulator's modular architecture, built on six pluggable abstract base classes, enables rapid experimentation with different scheduling algorithms, routing strategies, and interference models. We validated the WiFi model through nine internal experiments (all matching analytical predictions exactly), external reproduction of Bianchi's published results (matching within 0.003%), and cross-validation against ns-3 packet-level simulations, establishing self-consistency, fidelity to the canonical analytical framework, and agreement with packet-level ground truth.

The key finding enabled by `ncsim` is that wireless interference causes not merely inaccurate performance predictions but *systematically wrong scheduler choices*. In a 108-run factorial comparison across three schedulers, three network sizes, and three DAG complexities, the scheduler that appeared optimal under an interference-free model was dominated under realistic CSMA/CA conditions in 27.8% of scenarios, with performance regret reaching 168%. The sophisticated HEFT scheduler, which dominates under interference-free evaluation, was outperformed by a simple round-robin baseline in 4 of 6 large-network scenarios. Sensitivity analyses further revealed that interference impact peaks at moderate communication-to-computation ratios and amplifies super-linearly with concurrent workflows—conditions that characterize the wireless edge deployments that motivate this research.

The rank inversion finding suggests that the next generation of DAG schedulers for wireless edge computing should be *interference-aware*: the scheduler should account for wireless contention when making placement decisions, not merely optimize the critical path assuming uncontested bandwidth. Concrete design directions include querying the conflict graph during placement to penalize dense contention neighborhoods, two-phase scheduling that first places tasks then iteratively reassigns those contributing most to contention, and interference "budgets" that limit concurrent cross-node transfers. More broadly, DAG scheduling evaluations that assume uncontested links risk systematically wrong conclusions about which algorithms are superior. `ncsim` enables researchers to incorporate realistic wireless modeling at low cost—a single 108-run comparison completes in under 4 minutes—avoiding these systematic blind spots.

These findings underscore the value of tools like `ncsim` that make realistic wireless modeling accessible without the computational overhead of packet-level simulation. We identify several specific directions for future work: (1) developing interference-aware scheduling heuristics that query the conflict graph during task placement to avoid generating dense contention neighborhoods; (2) dynamic rescheduling that adjusts task placements in response to observed network congestion during execution; (3) incorporating node mobility with time-varying topology to model scenarios such as robotic edge computing; (4) multi-slot node execution to support GPU-like parallelism in heterogeneous edge deployments; (5) energy consumption models that account for the increased energy cost of contention-induced retransmissions; and (6) extending the ns-3 cross-validation (Section VI-C) to a wider range of traffic patterns and multi-hop topologies to further characterize the fidelity boundaries of the flow-level approximation.

The `ncsim` simulator and all experimental configurations used in this paper are available as open-source software to support reproducibility and further investigation.

Acknowledgment

This work was supported in part by Army Research Laboratory under Cooperative Agreement W911NF-17-2-0196.

References

[1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, Oct. 2016.

[2] ns-3 Consortium, "ns-3: A Discrete-Event Network Simulator," https://www.nsnam.org/, accessed Mar. 2026.

[3] A. Varga and R. Hornig, "An Overview of the OMNeT++ Simulation Environment," in *Proc. ICST SIMUTools*, 2008.

[4] H. Casanova, A. Giersch, A. Legrand, M. Quinson, and F. Suter, "Versatile, Scalable, and Accurate Simulation of Distributed Applications and Platforms," *Journal of Parallel and Distributed Computing*, vol. 74, no. 10, pp. 2899–2917, 2014.

[5] H. Casanova, S. Pandey, J. Oeth, R. Tanaka, F. Suter, and R. Ferreira da Silva, "WRENCH: A Framework for Simulating Workflow Management Systems," in *Proc. WORKS Workshop*, 2018, pp. 74–85.

[6] R. N. Calheiros, R. Ranjan, A. Beloglazov, C. A. F. De Rose, and R. Buyya, "CloudSim: A Toolkit for Modeling and Simulation of Cloud Computing Environments and Evaluation of Resource Provisioning Algorithms," *Software: Practice and Experience*, vol. 41, no. 1, pp. 23–50, 2011.

[7] H. Gupta, A. Vahid Dastjerdi, S. K. Ghosh, and R. Buyya, "iFogSim: A Toolkit for Modeling and Simulation of Resource Management Techniques in the Internet of Things, Edge and Fog Computing Environments," *Software: Practice and Experience*, vol. 47, no. 9, pp. 1275–1296, 2017.

[8] C. Sonmez, A. Ozgovde, and C. Ersoy, "EdgeCloudSim: An Environment for Performance Evaluation of Edge Computing Systems," *Transactions on Emerging Telecommunications Technologies*, vol. 29, no. 11, e3493, 2018.

[9] G. Bianchi, "Performance Analysis of the IEEE 802.11 Distributed Coordination Function," *IEEE Journal on Selected Areas in Communications*, vol. 18, no. 3, pp. 535–547, Mar. 2000.

[10] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, 2002.

[11] T. L. Adam, K. M. Chandy, and J. R. Dickson, "A Comparison of List Schedules for Parallel Processing Systems," *Communications of the ACM*, vol. 17, no. 12, pp. 685–690, 1974.

[12] Y.-K. Kwok and I. Ahmad, "Static Scheduling Algorithms for Allocating Directed Task Graphs to Multiprocessors," *ACM Computing Surveys*, vol. 31, no. 4, pp. 406–471, 1999.

[13] G. C. Sih and E. A. Lee, "A Compile-Time Scheduling Heuristic for Interconnection-Constrained Heterogeneous Processor Architectures," *IEEE Trans. Parallel Distrib. Syst.*, vol. 4, no. 2, pp. 175–187, 1993.

[14] J. Coleman and B. Krishnamachari, "Comparing Task Graph Scheduling Algorithms: An Adversarial Approach," arXiv preprint arXiv:2403.07120, 2024.

[15] J. Coleman, B. Krishnamachari, and contributors, "SAGA: Scheduling Algorithms Gathered (software)," https://github.com/anrgusc/saga, 2024.

[16] J. Coleman and B. Krishnamachari, "Parameterized Task Graph Scheduling Algorithm for Comparing Algorithmic Components," arXiv preprint arXiv:2403.07112, 2024.

[17] P. Ghosh, Q. Nguyen, P. K. Sakulkar, J. A. Tran, A. Knezevic, J. Wang, Z. Lin, B. Krishnamachari, M. Annavaram, and S. Avestimehr, "Jupiter: A Networked Computing Architecture," in *Proc. UCC Companion*, 2021.

[18] A. Poylisher, A. Cichocki, K. Guo, J. Hunziker, L. Kant, B. Krishnamachari, S. Avestimehr, and M. Annavaram, "Tactical Jupiter: Dynamic Scheduling of Dispersed Computations in Tactical MANETs," in *MILCOM 2021*, 2021.

[19] X. Zhao, D. Hu, and B. Krishnamachari, "Design and Experimental Evaluation of Algorithms for Optimizing the Throughput of Dispersed Computing," arXiv preprint arXiv:2112.13875, 2021.

[20] S. Suryavansh, C. Bothra, K. T. Kim, M. Chiang, C. Peng, and S. Bagchi, "I-BOT: Interference-Based Orchestration of Tasks for Dynamic Unmanaged Edge Computing," arXiv preprint arXiv:2011.05925, 2020.

[21] X. Li, M. Abdallah, S. Suryavansh, M. Chiang, K. T. Kim, and S. Bagchi, "DAG-based Task Orchestration for Edge Computing," in *Proc. SRDS*, 2022, pp. 23–34.

[22] X. Li, M. Abdallah, Y.-Y. Lou, M. Chiang, K. T. Kim, and S. Bagchi, "Dynamic DAG-Application Scheduling for Multi-Tier Edge Computing in Heterogeneous Networks," arXiv preprint arXiv:2409.10839, 2024.

[23] S. Yi, Z. Hao, Q. Zhang, Q. Zhang, W. Shi, and Q. Li, "LAVEA: Latency-aware Video Analytics on Edge Computing Platform," in *Proc. SEC*, 2017.

[24] W. Zhang, S. Li, L. Liu, Z. Jia, Y. Zhang, and D. Raychaudhuri, "Hetero-Edge: Orchestration of Real-time Vision Applications on Heterogeneous Edge Clouds," in *IEEE INFOCOM*, 2019, pp. 1270–1278.

[25] L. Lin, P. Li, J. Xiong, and M. Lin, "Distributed and Application-aware Task Scheduling in Edge-clouds," arXiv preprint arXiv:1902.04362, 2019.

[26] J. Liang, K. Li, C. Liu, and K. Li, "Joint offloading and scheduling decisions for DAG applications in mobile edge computing," *Neurocomputing*, vol. 424, pp. 160–171, 2021.

[27] K. Li, "Energy-Constrained DAG Scheduling on Edge and Cloud Servers with Overlapped Communication and Computation," *Journal of Grid Computing*, 2024.

[28] L. Long, Z. Liu, J. Shen, and Y. Jiang, "SecDS: A security-aware DAG task scheduling strategy for edge computing," *Future Generation Computer Systems*, p. 107627, 2025.

[29] M. Chiang and T. Zhang, "Fog and IoT: An Overview of Research Opportunities," *IEEE Internet of Things Journal*, vol. 3, no. 6, pp. 854–864, 2016.

[30] M. Chiang, S. Ha, I Chih-Lin, F. Risso, and T. Zhang, "Clarifying Fog Computing and Networking: 10 Questions and Answers," *IEEE Communications Magazine*, vol. 55, no. 4, pp. 18–20, 2017.

[31] K. T. Kim, C. Joe-Wong, and M. Chiang, "Coded Edge Computing," in *IEEE INFOCOM*, 2020, pp. 237–246.

[32] F. Daneshgaran, M. Laddomada, F. Mesiti, M. Mondin, and M. Zanolo, "Saturation Throughput Analysis of IEEE 802.11 in the Presence of Non Ideal Transmission Channel and Capture Effects," *IEEE Trans. Communications*, vol. 56, no. 7, pp. 1167–1178, Jul. 2008.

[33] M. Zorzi and R. R. Rao, "Capture and Retransmission Control in Mobile Radio," *IEEE J. Sel. Areas Commun.*, vol. 12, no. 8, pp. 1289–1298, Oct. 1994.

[34] Z. Hadzi-Velkov and B. Spasenovski, "Capture Effect in IEEE 802.11 Basic Service Area Under Influence of Rayleigh Fading and Near/Far Effect," in *IEEE PIMRC*, 2002.